13.10.21 12:35

Tutorial
Neural Language Modelling

Jetic Gū

CMPT413/713

Overview

- Focus: Language Modelling
- Architecture: Any Neural Sequence Model
- Core Ideas:
    1. What is a Language Model?
    2. What is a neural language model?
    3. What can a neural language model learn?

P0
Review

What you have seen so far

- Statistical Language Models
- Word Embeddings and Bag-of-Words

Review

P1
LM

What is a Language Model?

- A mathematical model that models human language
  - Probability distribution over a sequence of words

$$P(W_{0,1,\ldots,n})$$

  - Statistical LM: N-gram (chain rule)

$$P(W_{0,1,\ldots,n}) = P(W_0) \times P(W_1 \mid W_0) \times P(W_2 \mid W_{0,1}) \times \ldots$$

    - N-gram LM is an oversimplification of the original distribution we are trying to compute
    - Advantage: good for a number of tasks: Syntactic Parsing/Tagging, Generation

Concept

P1
LM

What is a Language Model?

- A mathematical model that models human language
  - Probability distribution over a sequence of words

$$P(W_{0,1,\ldots,n})$$

  - Statistical LM: N-gram (chain rule)

$$P(W_{0,1,\ldots,n}) = P(W_0) \times P(W_1 \mid W_0) \times P(W_2 \mid W_{0,1}) \times \ldots$$

    - N-gram LM is an oversimplification of the original distribution we are trying to compute
    - Advantage: good for a number of tasks: Syntactic Parsing/Tagging, Generation

Concept

Neural LM

- **BAD**: Bag-of-Words Language Model
- **OK**: Vanilla NLM using RNN
- **Good**: NLM using Gated RNN and Advanced Objectives
- **Very Good**: NLM using Attention and Advanced Objectives
- A neural LM is also an encoder, which generates feature representation for natural language input

Concept

Bag-of-Words Language Model

- Compute the probability of a sentence by calculating BoW representation of all word embeddings in the sentence, then feed into an MLP
  - Embedding of word w_i is $E(w_i)$
  - A sentence $S = [w_0, w_1, \ldots, w_{n-1}]$
  - Bag of Words: $BoW(S) = \sum_i E(w_i)$
  - Objective?

Concept

Bag-of-Words Language Model

- Objective 1: generative
  - Generative LM gives the probability of the next word in a sentence, given all previous words
  - e.g. $P(w_i = \text{cheese} \mid w_{<i} = \text{I want to eat})$
- Objective 2: discriminative
  - Discriminative LM gives probability over different classes for a sentence
  - e.g. sentiment analysis
    $P(class = \text{positive} \mid w_{<i} = \text{I want to eat cheese})$

Concept

Bag-of-Words Language Model

- Objective 1: generative
  - Generative LM gives the probability of the next word in a sentence, given all previous words
  - e.g. $P(w_i = \text{cheese} \mid w_{<i} = \text{I want to eat})$

Concept

P2
NLM

Bag-of-Words Language Model

- Objective 1: generative
  - Generative LM gives the probability of the next word in a sentence, given all previous words
  - e.g. $P(w_i = \text{cheese} \mid w_{<i} = \text{I want to eat})$

$P(w = \text{I} \mid \text{""}) = MLP(h_0)[\text{I}]$

Concept

Bag-of-Words Language Model

- Objective 1: generative
  - Generative LM gives the probability of the next word in a sentence, given all previous words
  - e.g. $P(w_i = \text{cheese} \mid w_{<i} = \text{I want to eat})$

$P(w = \text{I} \mid \text{""}) = MLP(h_0)[\text{I}]$

$P(w = \text{want} \mid \text{I}) = MLP(E(\text{I}))[\text{want}]$

Concept

Bag-of-Words Language Model

- Objective 1: generative
  - Generative LM gives the probability of the next word in a sentence, given all previous words
  - e.g. $P(w_i = \text{cheese} \mid w_{<i} = \text{I want to eat})$

$P(w = \text{I} \mid \text{""}) = MLP(h_0)[\text{I}]$

$P(w = \text{want} \mid \text{I}) = MLP(E(\text{I}))[\text{want}]$

$P(w = \text{to} \mid \text{I want}) = MLP(E(\text{I}) + E(\text{want}))[\text{to}]$

Concept

Bag-of-Words Language Model

- Objective 1: generative
  - Generative LM gives the probability of the next word in a sentence, given all previous words
  - e.g. $P(w_i = \text{cheese} \mid w_{<i} = \text{I want to eat})$

$P(w = \text{I} \mid \text{""}) = MLP(h_0)[\text{I}]$

$P(w = \text{want} \mid \text{I}) = MLP(E(\text{I}))[\text{want}]$

$P(w = \text{to} \mid \text{I want}) = MLP(E(\text{I}) + E(\text{want}))[\text{to}]$

$P(w = \text{eat} \mid \text{I want to}) = MLP(E(\text{I}) + E(\text{want}) + E(\text{to}))[\text{eat}]$

Concept

P2
NLM

Bag-of-Words Language Model

- Objective 1: generative
  - Generative LM gives the probability of the next word in a sentence, given all previous words
  - e.g. $P(w_i = \text{cheese} \mid w_{<i} = \text{I want to eat})$

$P(w = \text{I} \mid \text{""}) = MLP(h_0)[\text{I}]$

$P(w = \text{want} \mid \text{I}) = MLP(E(\text{I}))[\text{want}]$

$P(w = \text{to} \mid \text{I want}) = MLP(E(\text{I}) + E(\text{want}))[\text{to}]$

$P(w = \text{eat} \mid \text{I want to}) = MLP(E(\text{I}) + E(\text{want}) + E(\text{to}))[\text{eat}]$

$P(w = \text{cheese} \mid \text{I want to eat}) = MLP(E(\text{I}) + E(\text{want}) + E(\text{to}) + E(\text{eat}))[\text{cheese}]$

Concept

P2
NLM

Bag-of-Words for Generative

- Training data:
  $D = [S_0, S_1, \dots]$, where $S_i = [\overline{w}_{i,0}, \overline{w}_{i,1}, \dots]$

- Output layer
  $P(w_i \mid w_{<i}) = o = MLP(BoW(w_{<i})) \in |V|$

- Training:
  $$Loss = \sum_{S \in D} \sum_{i=0}^{i<|S|} -\log(P(w_i = \overline{w}_i \mid \overline{w_{<i}}))$$

Concept

Bag-of-Words for Discriminative

- Training data:
  $D = [(S_0, \overline{c}_0), (S_1, \overline{c}_1), \dots]$, where $S_i = [\overline{w}_{i,0}, \overline{w}_{i,1}, \dots], \overline{c} \in C$

- Output layer
  $P(c \mid S) = o = MLP(BoW(w, w \in S)) \in |C|$

- Training:
  $$Loss = \sum_{S_i, \overline{c}_i \in D} -\log(P(c = \overline{c}_i \mid S_i))$$

Why is BoW not a good LM?

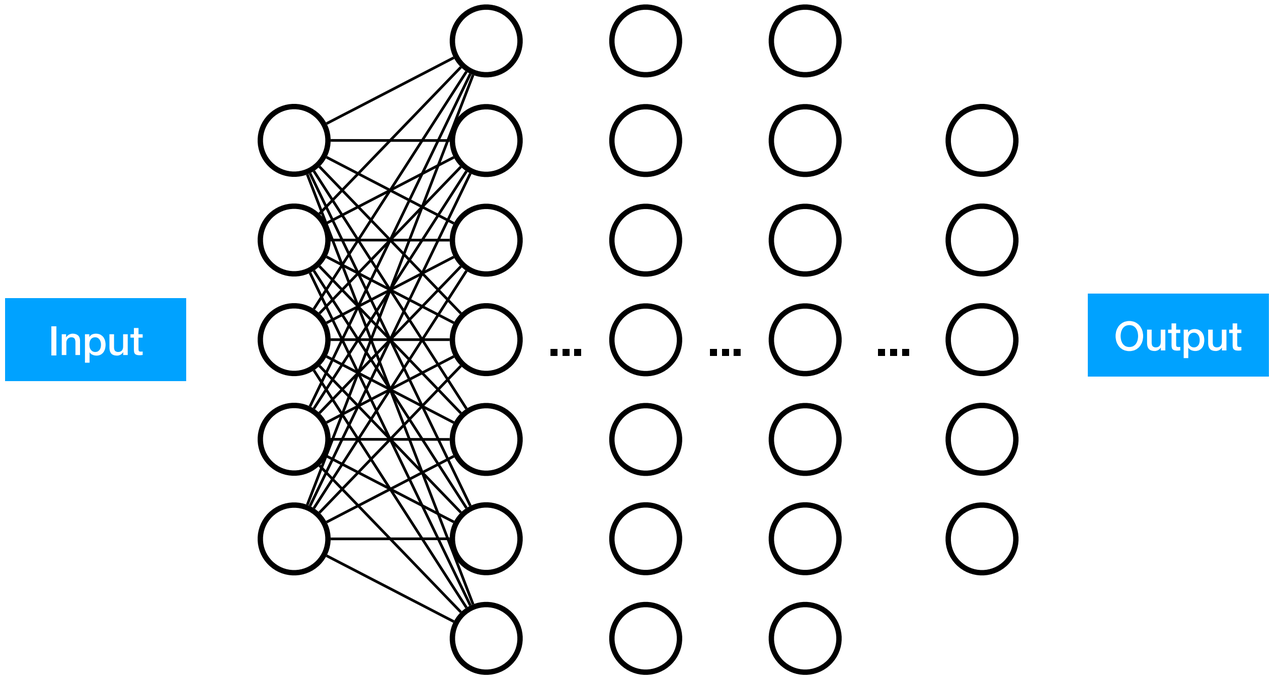
- No regard for word order
  "*Frank is madly in love with Mary*" and
  "*Mary is madly in love with Frank*" would have the same BoW

Concept

P2
NLM

Recurrent Neural Network

- So far in the world of MLP



Review

P2
NLM

Recurrent Neural Network

- So far in the world of MLP
- Given input x as a vector

$$h_0 = \sigma(W_0 x + b_0)$$

$$h_1 = \sigma(W_1 h_0 + b_1)$$

...

$$o = \sigma(W_n h_{n-1} + b_n)$$

- Both x and o are fixed length, not suitable for variable length natural language

Review

P2
NLM

Recurrent Neural Network

Concept

P2
NLM

Recurrent Neural Network

- Vanilla RNN

Concept

P2
NLM

Recurrent Neural Network

- Vanilla RNN
  - For input sequence $X = x_0, x_1, \ldots x_{n-1}$

Concept

P2
NLM

Recurrent Neural Network

- Vanilla RNN
  - For input sequence $X = x_0, x_1, \ldots x_{n-1}$
  - Vector h_{i-1} holds information about $x_0, \ldots, x_{i-1}$

Concept

Recurrent Neural Network

- Vanilla RNN
  - For input sequence $X = x_0, x_1, \ldots x_{n-1}$
  - Vector h_{i-1} holds information about $x_0, \ldots, x_{i-1}$
  - $h_i = RNN(h_{i-1}, x_i) = MLP([h_{i-1}; x_i])$
    where $[a; b]$ is the concatenation of vectors a and b

Concept

P2
NLM

Recurrent Neural Network

- Vanilla RNN
  - For input sequence $X = x_0, x_1, \ldots x_{n-1}$
  - Vector h_{i-1} holds information about $x_0, \ldots, x_{i-1}$
  - $h_i = RNN(h_{i-1}, x_i) = MLP([h_{i-1}; x_i])$
    where $[a; b]$ is the concatenation of vectors a and b
  - h_0 is part of your parameter that gets trained with the rest of your model

Concept

RNN Encodes a Sentence

- Vanilla RNN encodes a sentence $S = w_0, w_1, w_2, \ldots, w_{n-1}$

RNN Encodes a Sentence

- Vanilla RNN encodes a sentence $S = w_0, w_1, w_2, \ldots, w_{n-1}$

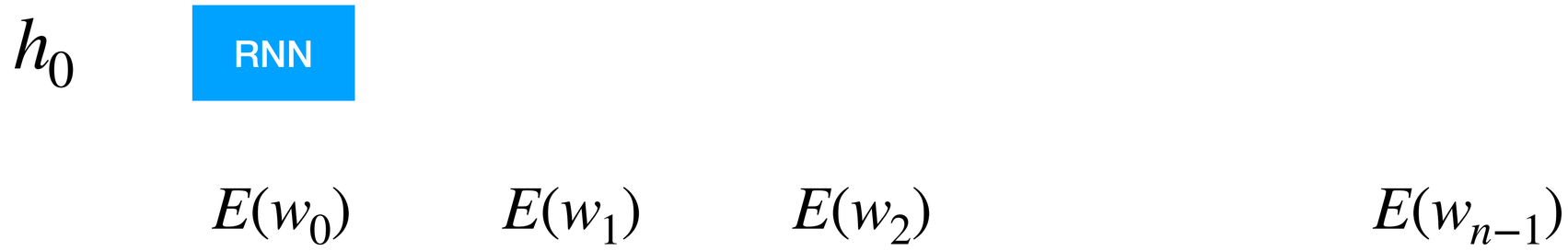
RNN

RNN Encodes a Sentence

- Vanilla RNN encodes a sentence $S = w_0, w_1, w_2, \ldots, w_{n-1}$

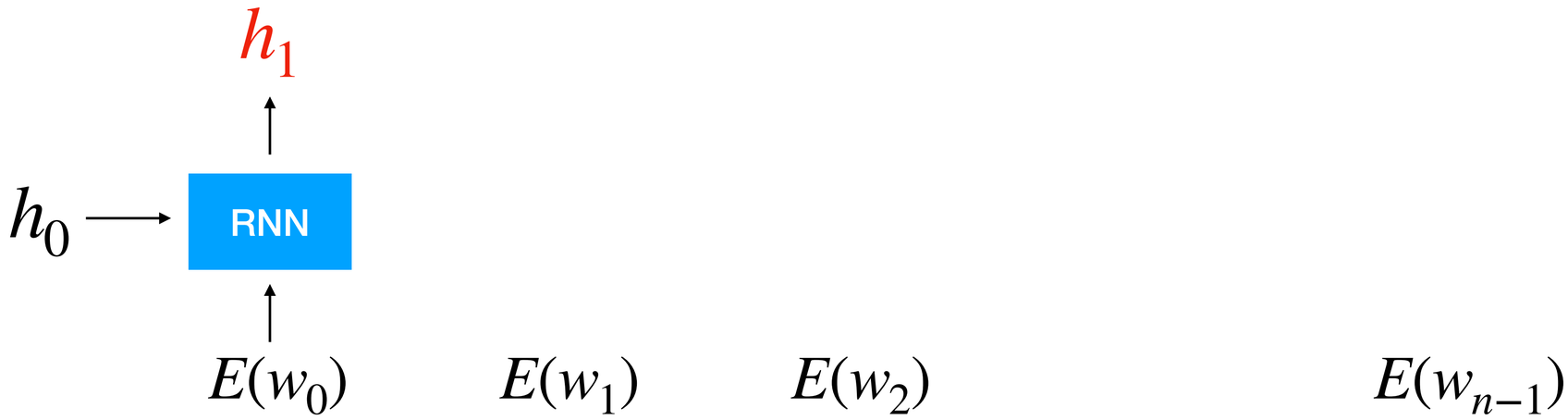
RNN Encodes a Sentence

- Vanilla RNN encodes a sentence $S = w_0, w_1, w_2, \ldots, w_{n-1}$



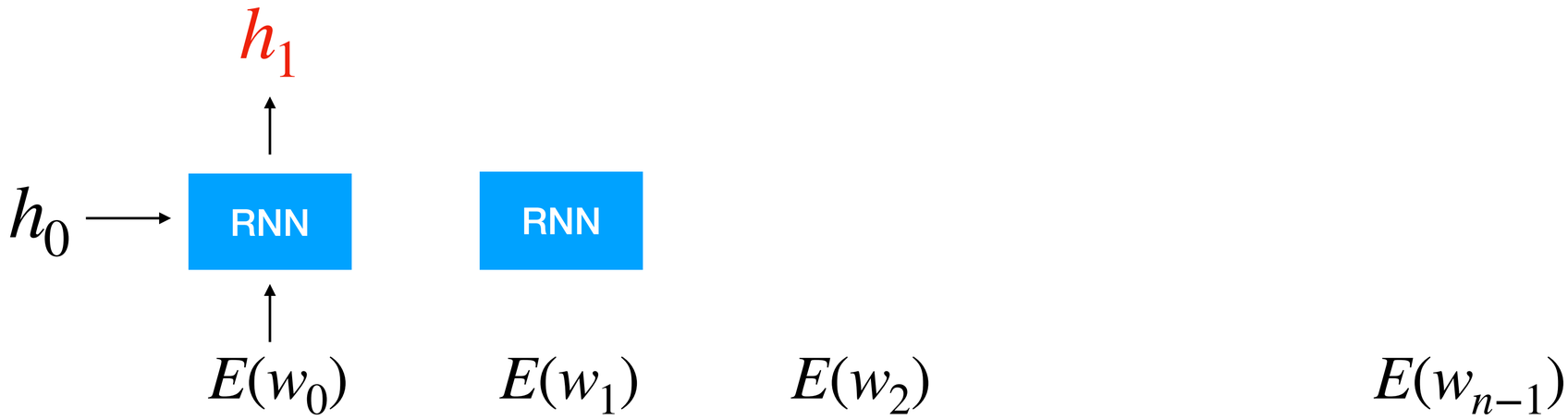
RNN Encodes a Sentence

- Vanilla RNN encodes a sentence $S = w_0, w_1, w_2, \ldots, w_{n-1}$



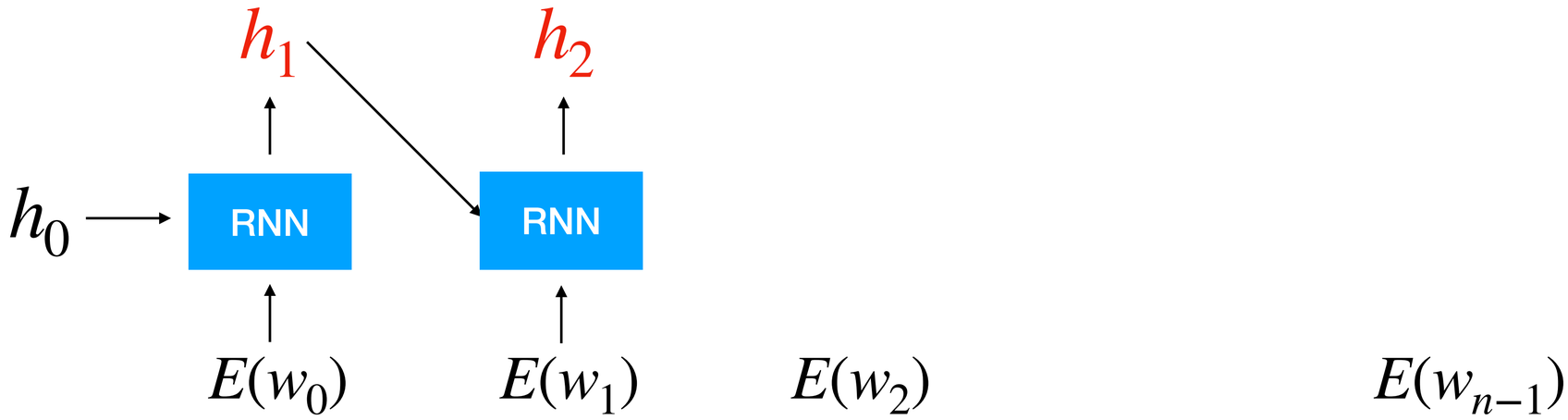
RNN Encodes a Sentence

- Vanilla RNN encodes a sentence $S = w_0, w_1, w_2, \ldots, w_{n-1}$



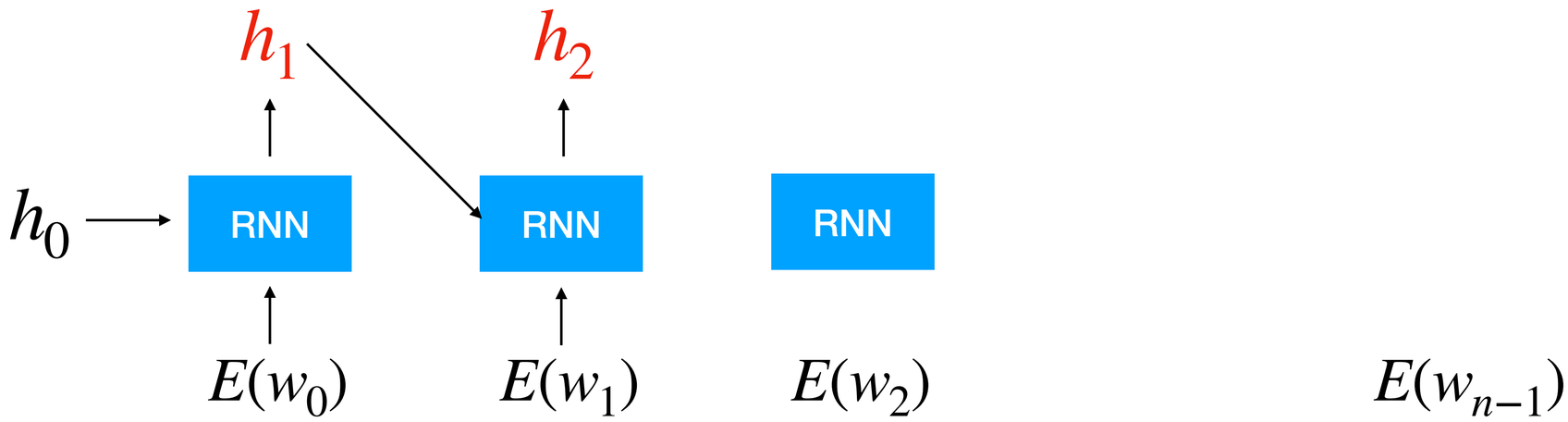
RNN Encodes a Sentence

- Vanilla RNN encodes a sentence $S = w_0, w_1, w_2, \ldots, w_{n-1}$



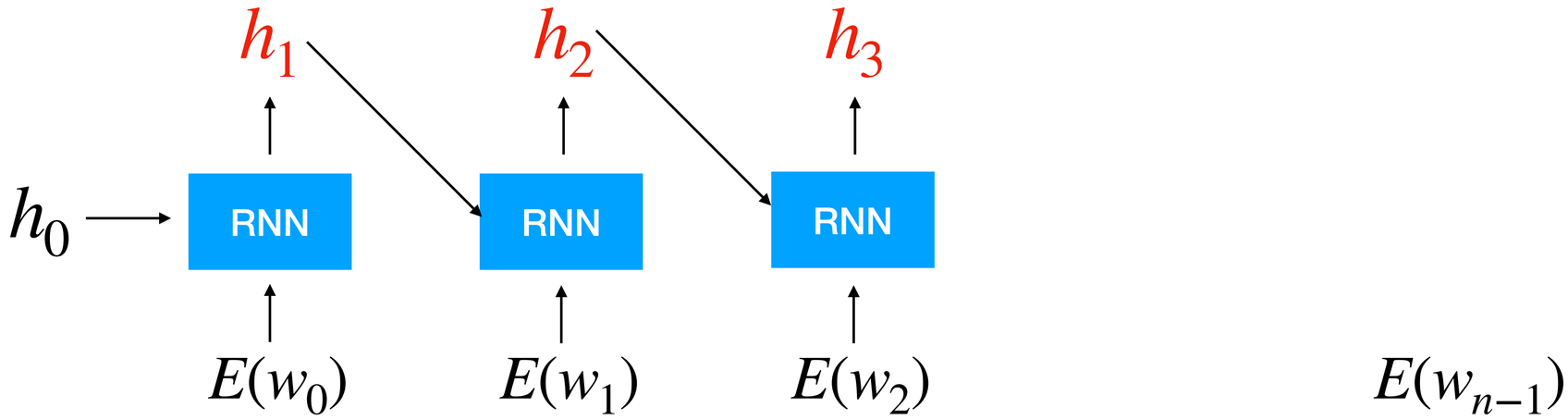
RNN Encodes a Sentence

- Vanilla RNN encodes a sentence $S = w_0, w_1, w_2, \ldots, w_{n-1}$



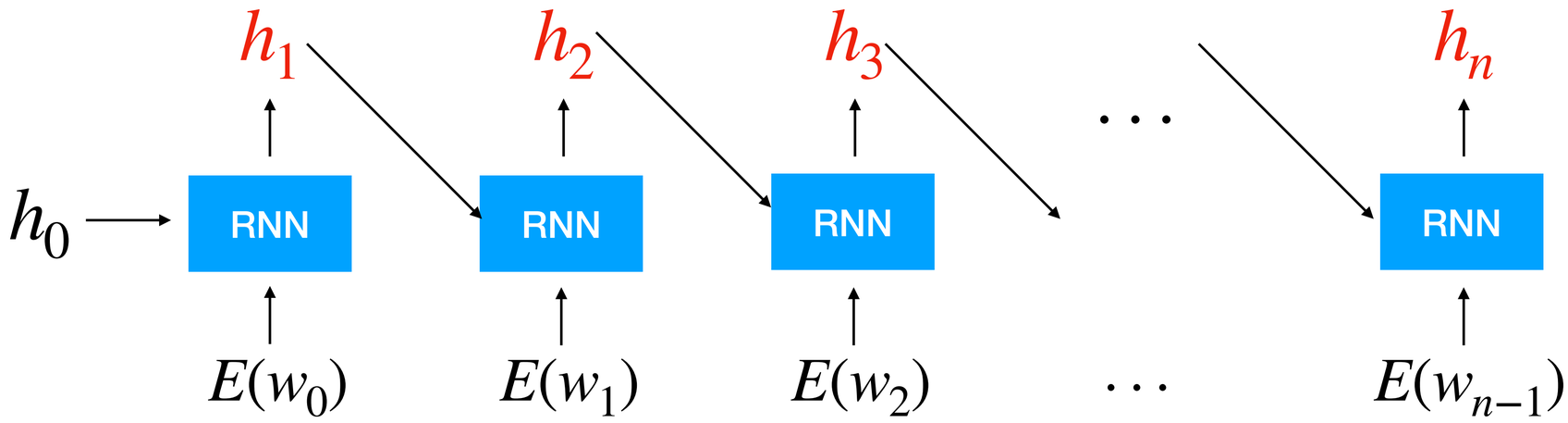
RNN Encodes a Sentence

- Vanilla RNN encodes a sentence $S = w_0, w_1, w_2, \ldots, w_{n-1}$



RNN Encodes a Sentence

- Vanilla RNN encodes a sentence $S = w_0, w_1, w_2, \ldots, w_{n-1}$



P2
NLM

RNN for Generative

Concept

RNN for Generative

- Training data:
  $D = [S_0, S_1, \dots]$, where $S_i = [\overline{w}_{i,0}, \overline{w}_{i,1}, \dots]$

RNN for Generative

- Training data:
  $D = [S_0, S_1, \dots]$, where $S_i = [\overline{w}_{i,0}, \overline{w}_{i,1}, \dots]$

- Output layer
  $P(w_i \mid w_{<i}) = o = MLP(h_i) \in |V|$, where $h_i = RNN(h_{i-1}, w_{i-1})$

Concept

RNN for Generative

- Training data:
  $D = [S_0, S_1, \dots]$, where $S_i = [\overline{w}_{i,0}, \overline{w}_{i,1}, \dots]$

- Output layer
  $P(w_i \mid w_{<i}) = o = MLP(h_i) \in |V|$, where $h_i = RNN(h_{i-1}, w_{i-1})$

- Training:
  $$Loss = \sum_{S \in D} \sum_{i=0}^{i<|S|} -\log(P(w_i = \overline{w}_i \mid \overline{w_{<i}}))$$

Concept

P2
NLM

Bag-of-Words for Discriminative

Concept

Bag-of-Words for Discriminative

- Training data:
  $D = [(S_0, c_0), (S_1, c_1), \dots]$, where $S_i = [\overline{w}_{i,0}, \overline{w}_{i,1}, \dots], c \in C$

Bag-of-Words for Discriminative

- Training data:
  $D = [(S_0, c_0), (S_1, c_1), \dots]$, where $S_i = [\overline{w}_{i,0}, \overline{w}_{i,1}, \dots], c \in C$

- Output layer
  $P(c \mid S) = o = MLP(h_{|S|}) \in |C|$, , where $h_i = RNN(h_{i-1}, w_i)$

Concept

Bag-of-Words for Discriminative

- Training data:
  $D = [(S_0, c_0), (S_1, c_1), \dots]$, where $S_i = [\overline{w}_{i,0}, \overline{w}_{i,1}, \dots], c \in C$

- Output layer
  $P(c \mid S) = o = MLP(h_{|S|}) \in |C|$, , where $h_i = RNN(h_{i-1}, w_i)$

- Training:
  $$Loss = \sum_{S_i, \overline{c}_i \in D} -\log(P(c = \overline{c}_i \mid S_i))$$

RNN was use for

- Sentiment Analysis/Text Classification
- Syntactic Tagging/Parsing
- Sequence-to-Sequence Training

Concept

State-of-the-Art RNN LM

1. CL2014008 [Cho et al.] Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation
2. ML1997001 [Hochreiter et Schmidhuber] Long short-term memory
3. CL2018460 [Devlin et al.] BERT Pre-training of Deep Bidirectional Transformers for Language Understanding

Concept

State-of-the-Art RNN LM

- Instead of vanilla RNN, uses gated RNN

1. CL2014008 [Cho et al.] Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation
2. ML1997001 [Hochreiter et Schmidhuber] Long short-term memory
3. CL2018460 [Devlin et al.] BERT Pre-training of Deep Bidirectional Transformers for Language Understanding

Concept

State-of-the-Art RNN LM

- Instead of vanilla RNN, uses gated RNN
  - Gated Recurrent Unit[1]

1. CL2014008 [Cho et al.] Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation
2. ML1997001 [Hochreiter et Schmidhuber] Long short-term memory
3. CL2018460 [Devlin et al.] BERT Pre-training of Deep Bidirectional Transformers for Language Understanding

Concept

State-of-the-Art RNN LM

- Instead of vanilla RNN, uses gated RNN
  - Gated Recurrent Unit[1]
  - Long- Short-Term Memory[2]

1. CL2014008 [Cho et al.] Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation
2. ML1997001 [Hochreiter et Schmidhuber] Long short-term memory
3. CL2018460 [Devlin et al.] BERT Pre-training of Deep Bidirectional Transformers for Language Understanding

Concept

State-of-the-Art RNN LM

- Instead of vanilla RNN, uses gated RNN
  - Gated Recurrent Unit[1]
  - Long- Short-Term Memory[2]
- Uses more complex pre-training objectives such as masking and denoising

1. CL2014008 [Cho et al.] Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation
2. ML1997001 [Hochreiter et Schmidhuber] Long short-term memory
3. CL2018460 [Devlin et al.] BERT Pre-training of Deep Bidirectional Transformers for Language Understanding

Concept

State-of-the-Art RNN LM

- Instead of vanilla RNN, uses gated RNN
  - Gated Recurrent Unit[1]
  - Long- Short-Term Memory[2]
- Uses more complex pre-training objectives such as masking and denoising
- RNN-based LM was surpassed only by Multi-Headed Attention Encoders[3], and RNN is still very good in production due to its simplicity

1. CL2014008 [Cho et al.] Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation
2. ML1997001 [Hochreiter et Schmidhuber] Long short-term memory
3. CL2018460 [Devlin et al.] BERT Pre-training of Deep Bidirectional Transformers for Language Understanding

Concept

P3
NLM

What can NLM learn

Concept

What can NLM learn

- Research has shown that pre-trained RNN language models can capture representations that is beneficial to

Concept

What can NLM learn

- Research has shown that pre-trained RNN language models can capture representations that is beneficial to
  - POS Tagging, Named Entity Recognition, Text classification

Concept

What can NLM learn

- Research has shown that pre-trained RNN language models can capture representations that is beneficial to
  - POS Tagging, Named Entity Recognition, Text classification
  - With supervision: Translation, Summarisation, etc.

Concept

What can NLM learn

- Research has shown that pre-trained RNN language models can capture representations that is beneficial to
  - POS Tagging, Named Entity Recognition, Text classification
  - With supervision: Translation, Summarisation, etc.
  - Does pre-trained LM capture semantics?

Concept